

Agentic Accessibility Auditor (Axion): An AI-Driven Pipeline for Automated Android Mobile UI Accessibility Auditing

Muhammad Noor ul Haq
NCRA-UAV Dependability Lab
FAST NUCES, Islamabad
Islamabad, Pakistan
i232520@isb.nu.edu.pk

Salar Khan
NCRA-UAV Dependability Lab
FAST NUCES, Islamabad
Islamabad, Pakistan
i232607@isb.nu.edu.pk

Ayesha Naveed
NCRA-UAV Dependability Lab
FAST NUCES, Islamabad
Islamabad, Pakistan
i233068@isb.nu.edu.pk

Abstract—Accessibility evaluation of Android mobile applications remains largely manual, slow, and inconsistent, and existing automated tools detect only a fraction of the structural issues that affect users with visual, motor, auditory, or cognitive impairments. This paper presents the *Agentic Accessibility Auditor*, branded Axion, a working prototype developed over an eight-week summer internship at FAST-NUCES that combines an Android UI screenshot with its UIAutomator XML hierarchy to automatically detect accessibility violations, map them to WCAG 2.2 AA-aligned guidelines, and generate developer-facing explanations and fixes using a large language model (LLM) agent. The system is implemented as a six-layer pipeline – input, parsing, rule checking, agentic explanation, report generation, and evaluation – exposed through a FastAPI backend and a React/Tailwind web dashboard with authentication and per-user audit history. A deterministic rule engine implements 30 detection rules (R01–R30) mapped to 30 accessibility guidelines (G01–G30). Two supplementary computer-vision modules were trained and integrated: a YOLO11-based screenshot-only UI-element detector that provides a fallback when no XML hierarchy is available, and a Swin-Tiny-backboned multi-label crop classifier that confirms six pixel-dependent rules from rendered screenshots. The system was developed and tuned on the 7,068-screen MASC dataset and evaluated, without further tuning, on a disjoint 1,698-screen Rico holdout set, on which the rule engine reports a mean of 31.7 violations per screen and a mean accessibility score of 32.0/100, and the YOLO detector achieves mAP50 of 0.2546 (up from a zero-shot baseline of 0.0258), while the crop classifier reaches a Rico holdout macro-F1 of 0.22, the best result across a 33-backbone comparison sweep. We describe the full system design, dataset strategy, evaluation methodology, and empirical results, and close with a discussion of limitations and directions for future work.

Index Terms—accessibility testing, Android, WCAG, agentic AI, large language models, UI automation, YOLO, vision transformers, software engineering, human–computer interaction

I. INTRODUCTION

Mobile applications are expected to be usable by people with a wide range of abilities, yet accessibility evaluation in industry practice is still predominantly manual, ad hoc, and dependent on the reviewer’s familiarity with the Web Con-

tent Accessibility Guidelines (WCAG). Screen-reader walkthroughs and manual checklists do not scale to the pace of continuous mobile development, and purely visual (screenshot-only) automated tools cannot reliably detect structural issues such as missing content descriptions, duplicate accessible labels, undersized touch targets, or unlabeled input fields, because these properties are encoded in the underlying UI hierarchy rather than in pixels alone. Existing automated scanners typically apply a small, fixed rule set and stop at a raw diagnostic code, leaving a developer with no explanation of *why* an issue matters to a real user or *how* to fix it in their own layout XML.

This project set out to close that gap by treating Android accessibility auditing as a two-input task: a rendered **screenshot** and its corresponding **UIAutomator XML** view hierarchy – the same structured tree that Android’s own TalkBack screen reader consumes. By parsing the XML into a structured component representation and applying a deterministic rule engine, the system detects concrete, reproducible violations. An *agentic* large language model (LLM) layer then explains each rule-detected violation in plain English, describes its impact on users with disabilities, and proposes a concrete developer fix – but is explicitly and architecturally constrained never to invent violations that the rule engine did not detect. The end result is packaged into an HTML/PDF audit report and surfaced through a branded web dashboard, **Axion**, complete with user accounts and a persistent history of past audits.

The project was carried out by a three-person intern team over an eight-week internship, with each member owning a primary pipeline layer while collaborating on shared artifacts: the rule engine, the JSON schema contracts that let the three layers integrate independently, and the requirements/design documentation. What began as a Must-Have/Should-Have/Stretch-tiered plan concluded with every tier delivered, including two originally-optional computer-vision models that were fully trained, evaluated on unseen data, and wired into the production backend in the project’s

final week. This report documents the objectives, system architecture, implementation, dataset and evaluation strategy, and the empirical results obtained on both the development dataset and an unseen holdout set, followed by a discussion of limitations and future work.

II. BACKGROUND

WCAG 2.2 AA. The project’s 30 accessibility guidelines (G01–G30) are each aligned to a specific WCAG 2.2 success criterion – for example G01 (missing accessible label) to 4.1.2 Name/Role/Value, G04 (small touch target) to 2.5.5 Target Size, and G09 (low contrast) to 1.4.3 Contrast (Minimum) – so every finding in the tool carries a traceable standards reference rather than an ad hoc justification.

UIAutomator and the Android accessibility tree. Android exposes a structured view hierarchy via `UIAutomator`, in which every node carries properties such as `class`, `text`, `content-desc`, `clickable`, `enabled`, `focusable`, `resource-id`, and `pixel bounds`. This tree is exactly what a screen reader relies on to announce and navigate a screen, which makes it the most direct available signal for structural accessibility problems – but it is unavailable for apps that intentionally suppress or obfuscate their accessibility tree, motivating the project’s screenshot-only fallback module.

Agentic LLM layers. A growing body of recent work applies LLMs directly as the accessibility-error *detector* on mobile UIs – ScreenAudit [9] prompts an LLM over screen-reader traversal output to find accessibility errors, and In-sight [10] uses an LLM to interpret and narrate live screens for blind and low-vision users. This project takes a deliberately different position: a language model is most reliable as an *explainer* layered over a deterministic source of truth rather than as the detector itself. This separation was adopted as a hard design constraint from the outset: the rule engine is the sole detector of violations, deterministic and reproducible by construction, while the LLM agent is restricted to explaining, contextualizing, and recommending fixes for issues it is handed, with an explicit anti-hallucination check verifying that every explanation returned by the model corresponds to a real violation ID that was actually sent to it.

III. PROBLEM STATEMENT AND OBJECTIVES

Android accessibility evaluation is manual, slow, and inconsistent, and existing automated tools miss structural issues that only the UI hierarchy exposes. There is a need for an automated, explainable, developer-friendly system that combines a screenshot, the UI hierarchy, deterministic rule checks, and AI-generated recommendations into a single audit workflow. The concrete objectives set for the internship were to:

- Accept a paired Android UI screenshot and UIAutomator XML file through a web interface;
- Parse the XML into structured component records (`class`, `bounds`, `text`, `content-desc`, `clickable`, `enabled`, `focusable`, `resource-id`);

- Implement deterministic rule-based accessibility checks for common Android UI issues;
- Map every detected issue to an accessibility guideline category with a WCAG reference and severity;
- Use an agentic explanation layer to produce plain-English explanations and developer-facing fixes, without inventing new issues;
- Generate an HTML/PDF audit report with an issue summary, affected elements, severity, and recommendations;
- Evaluate the tool on at least 25–40 Android screens and document the results.

Priorities were tiered into a Minimum Viable Product (parser → R01–R10 → LLM explanations → HTML/PDF report → Axion Upload/Dashboard/Report → authentication and per-user records → Docker deployment → evaluation on 25–40 screens), a Should-Have tier (R11–R20, dashboard filtering, batch processing), and a Stretch tier (R21–R30, a multi-label pixel-level crop classifier, a screenshot-only YOLO fallback detector, and interactive report highlighting). By project close, all three tiers – including both stretch computer-vision models – were implemented and wired into the production pipeline, and 99 of 99 tracked functional and non-functional requirements were marked Done in the project’s requirements traceability matrix.

IV. SYSTEM ARCHITECTURE

The system is structured as a six-layer pipeline in which every stage produces a versioned, schema-validated JSON artifact consumed by the next stage. This artifact-oriented design let the three interns develop and test their modules independently against agreed contracts rather than a shared in-memory state, and made regression testing tractable at dataset scale.

Figure 1 shows the logical data flow: a screenshot and UIAutomator XML pair enters the Input Layer; the Parsing Layer converts the XML into `components.json`; the Rule Checker (optionally confirmed by the crop classifier, or entirely substituted by the YOLO detector when XML is unavailable) produces `violations.json`; the Agentic Layer enriches each violation into `report.json`; the Report Layer renders `audit_report.html/.pdf`; and the Evaluation Layer aggregates results across the MASC and Rico holdout datasets.

At the physical level, the system is a FastAPI backend (`backend/`) exposing three route groups – `/api/v1/audit/*` for the pipeline, `/auth/*` for authentication, and `/records/*` for per-user history – behind a React 19 + Vite + Tailwind CSS frontend (`frontend/`), all orchestrated with Docker Compose (`backend`, `auditor`, and `frontend` services on an internal bridge network, `auditor-network`). Table I summarizes the technology stack.

V. DATASET STRATEGY

Two datasets anchor the project’s development-versus-generalization strategy, summarized in Table II. **MASC**

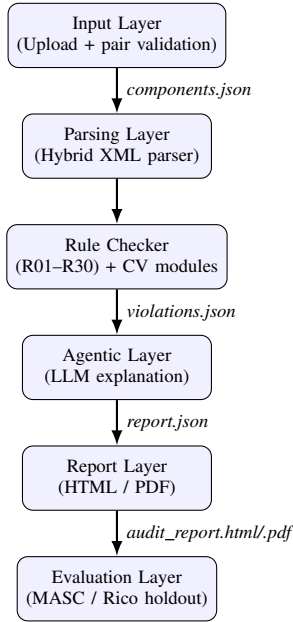


Fig. 1: Six-layer pipeline architecture. Each layer emits a schema-validated JSON artifact consumed by the next stage.

TABLE I: Technology stack summary.

Layer	Technology
Backend API	Python 3.11, FastAPI 0.110+
XML parsing	lxml / xml.etree.ElementTree
Image handling	PIL, OpenCV
Rule engine	Pure Python, no ML dependency
Object detection	Ultralytics YOLO11s (screenshot-only fallback)
Crop classification	timm swin_tiny_patch4_window7_224, PyTorch
Agent / LLM	Groq (llama-3.3-70b-versatile), with Anthropic/OpenAI/Gemini as alternates
Report rendering	Jinja2 (HTML) + Playwright/Chromium (PDF)
Frontend	React 19, Vite, Tailwind CSS, React Router
Auth	JWT (HS256), bcrypt, SMTP OTP, Google OAuth
Deployment	Docker, Docker Compose (bridge network)
Testing	pytest, FastAPI TestClient

(Mobile App Screens Classification dataset) [7] supplies 7,068 paired screenshot/XML screens across ten UI categories (chat, home, list, login, maps, menu, profile, search, settings, welcome), split 70%/15%/15% into train/val/test (4,943/1,056/1,069 screens respectively) and used for parser development, rule tuning, and all model training. **Rico holdout** [6], [8] is a 1,698-screen subset of the Rico UI corpus, kept strictly disjoint from MASC and reserved for final, unseen evaluation; it was never used to tune rules or train either computer-vision model, enforced structurally where possible – for instance, the YOLO dataset configuration for Rico points its train, val, and test splits all at the same held-out image set, so the model cannot accidentally be trained on it.

Supplementary real-device collection was also supported through the standard Android tool chain (`adb exec-out screencap`, `adb shell uiautomator dump`, `adb shell wm size/density`), for cases where a new, un-

TABLE II: Dataset strategy: MASC (development) vs. Rico holdout (unseen evaluation).

Dataset	Size	Split	Purpose
MASC	7,068 pairs	4,943 / 1,056 / 1,069 (train/val/test)	Development, parser tuning, rule development, model training
Rico holdout	1,698 pairs	100% held out	Final unseen evaluation – no tuning

catalogued app needed to be audited directly from an emulator or physical device.

VI. IMPLEMENTATION

A. Input Layer and Pair Validation

Axon’s Upload screen (Fig. 2) accepts a screenshot (PNG/JPG) and, optionally, a matching UIAutomator XML file through a sequential drop zone. A four-check Validation Status panel (screenshot detected, XML detected, valid pair, ready for analysis) gates the “Start Audit” action, and the backend independently re-validates that the two filenames share a stem or numeric screen ID before the pipeline runs, rejecting mismatched pairs with a clear error. The XML upload is optional: if it is missing, malformed, or parses to zero components, the pipeline transparently falls back to the screenshot-only YOLO detector described in Section VI-D1.

B. Parsing Layer: Hybrid XML Parser

`src/parser.py` implements a hybrid, single-traversal parser that recognizes four XML dialects encountered across the project’s datasets: native UIAutomator bounds syntax (`[l,t][r,b]`), MASC’s wrapper-nested widgets with numeric bounds, Rico’s space-separated bounds, and generic uploads. For every node it extracts the required fields (`class`, `text`, `content-desc`, `resource-id`, `clickable`, `enabled`, `focusable`, `bounds`) plus an extended set of fields discovered necessary during rule development – `focus_order` and `parent_id` (depth-first traversal order, used for focus-order and hierarchy rules), `long_clickable/scrollable` (gesture heuristics), `media_type`, `is_dialog`, and `hint/input_type/password/label_for` (input labelling). Malformed or incomplete XML is handled gracefully – null bytes are stripped before parsing, and missing bounds are mapped to `[0,0,0,0]` for UIAutomator-shaped nodes – so that a single bad file never crashes the batch pipeline. The parser output, `components.json`, conforms to a normative JSON Schema (`docs/schemas/auditor_schema.json`) validated automatically in the test suite.

C. Rule Checker: Deterministic Accessibility Engine

`src/rules.py` implements all 30 detection rules (R01–R30) as plain, side-effect-free functions chained by a single entry point, `check()`, which additionally drops components marked invisible before checking and reads a `device_info.dpi` value (default 160) to convert pixel

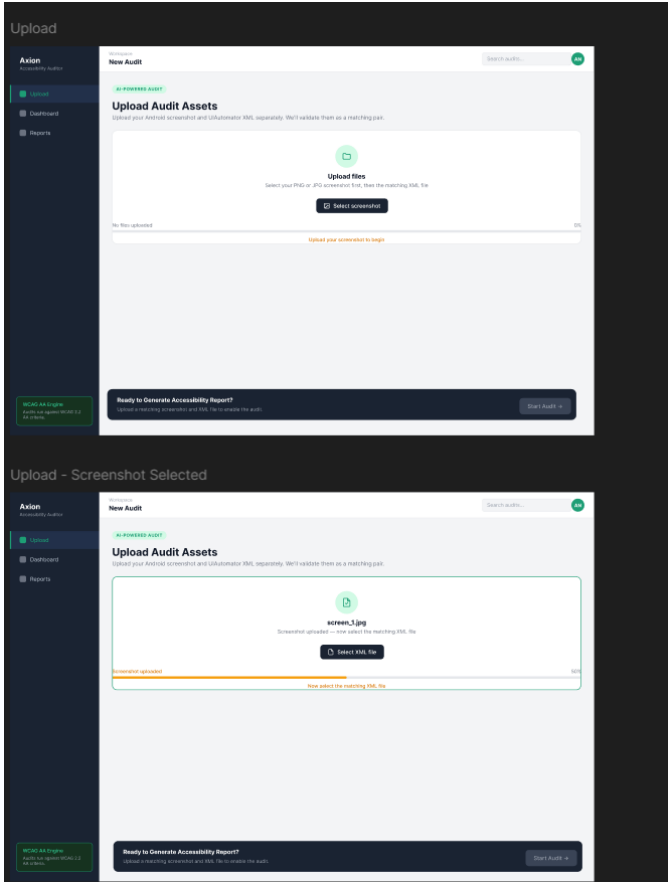


Fig. 2: Axion Upload screen: sequential screenshot/XML drop zone with live validation progress.

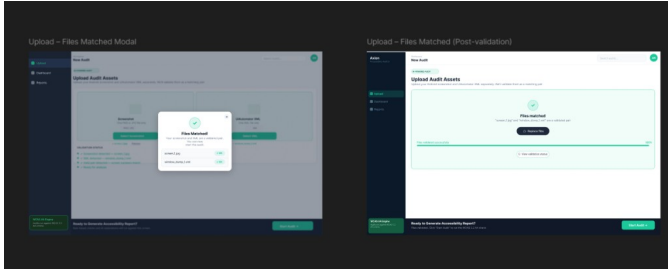


Fig. 3: Files Matched confirmation modal shown once a screenshot/XML pair validates, with the Validation Status checklist visible beneath.

bounds to density-independent pixels for the touch-target and spacing rules. Table III summarizes the ten Must-Have rules that formed the MVP; rules R11–R20 (Should-Have) and R21–R30 (Stretch) extend coverage to color-only information, missing captions, bad focus order, insufficient spacing, destructive actions without confirmation, vague error messages, missing screen titles, uncontrolled animation, and icon-only controls, among others.

Each violation record carries `rule_id`, `issue`, `component_id`, `class`, `bounds`, `guideline`, `severity` (Critical|High|Medium|Low, remapped to

TABLE III: Must-Have detection rules (R01–R10) mapped to guidelines and WCAG success criteria.

ID	Detection logic	Sev.	Guid.
R01	clickable + empty text + empty content-desc	High	G01
R02	Image/ImageButton clickable, empty content-desc	High	G02
R03	≥ 2 clickables share identical text/content-desc	Med	G03
R04	clickable width or height $< 48\text{dp}$	High	G04
R05	EditText with empty hint, text, content-desc	High	G05
R06	clickable, enabled=false, no explanation nearby	Med	G06
R07	zero/invalid bounds on an ally-relevant node	Med	G07
R08	bounding-box overlap $> 50\%$ of smaller element	Med	G08
R09	declared contrast $< 4.5:1$ text / $< 3:1$ icons	Med	G09
R10	TextView bounds below estimated text height	Low/Med	G10

Axion’s Critical/Serious/Moderate/Minor badges in the UI), and a recommendation; pair-based rules (R03, R08, R17) additionally attach a `related_component`. Listing 1 shows an abridged `violations.json` record for two rules on a single screen.

Listing 1: Abridged `violations.json` excerpt showing R02 and R05.

```
{
  "schema_version": "1.0",
  "screen_id": "screen_001",
  "total_violations": 2,
  "violations": [
    {
      "rule_id": "R02",
      "issue": "Image button without description",
      "component_id": "c_001",
      "class": "android.widget.ImageButton",
      "bounds": [32, 50, 80, 98],
      "guideline": "G02 – Image button without description",
      "severity": "High",
      "recommendation": "Add android:contentDescription with the action name, such as Back."
    },
    {
      "rule_id": "R05",
      "issue": "Unlabeled input field",
      "component_id": "c_002",
      "class": "android.widget.EditText",
      "bounds": [32, 120, 400, 168],
      "guideline": "G05 – Unlabeled input field",
      "severity": "High",
      "recommendation": "Add android:hint or a programmatic label linked via labelFor."
    }
  ]
}
```

The engine is deterministic by design constraint – identical input always yields an identical `violations.json` – which made regression testing against the full 7,068-screen MASC corpus and the 1,698-screen Rico holdout tractable. During the internship’s rule-accuracy pass, three confirmed false-positive sources and one parser bug were root-caused against real data rather than assumption and fixed:

- 1) **R07** was flagging non-interactive structural wrappers with zero bounds that a screen reader would never reach

- fixed with an `_is_ally_relevant` gate restricting R07 to clickable/focusable/labelled nodes;
- 2) **R08** was flagging a clickable scrollable container against its own clickable child rows (confirmed on a real Rico chat screen, where all 8 of 8 rows were falsely flagged) – fixed with an `_is_ancestor` helper that walks the `parent_id` chain and skips ancestor/descendant pairs, reducing R08 volume by **−43.2%** on the Rico holdout (12,738 → 7,238 violations);
- 3) **R30** was counting one violation per repeated list/grid-row icon instance – fixed with an `_dedupe_repeated` helper that collapses same-resource-id-and-bounds repeats into a single violation carrying an instance count;
- 4) the parser’s `_get_hint` helper only recognized the `hint/android:hint` attributes, while MASC’s real attribute name is `text-hint` – **R20** was structurally unable to ever fire (0 hits) until this alias was added, unblocking it to 749 hits on the full MASC corpus and, as a side effect, correcting an **R05** over-flagging pattern (2,117 → 717 hits, **−66%**).

All four fixes were verified against the complete pytest suite, a full MASC re-parse and re-check (7,068 screens, 0 failures), and a full Rico holdout re-evaluation (1,698 screens, 0 failures) – not just controlled fixtures.

D. Computer-Vision Modules

Two rendered-pixel signals were designed to complement the XML-only rule engine, both trained exclusively on MASC and evaluated – without any further tuning – on the disjoint Rico holdout set. Both modules run GPU-accelerated inference (CUDA) when a GPU is available – confirmed with `torch.cuda.is_available()` returning `True` inside the deployed container and `nvidia-smi` utilization observed during a live audit – and fall back to CPU otherwise, with device selection made explicit in both modules rather than left to an inference library’s implicit default.

1) *Screenshot-Only YOLO UI-Element Detector*: The rule engine’s dependency on a paired XML hierarchy is a hard limitation for apps that block or omit the accessibility tree. `src/yolo_ui_detector.py` addresses this with an Ultralytics YOLO11s object detector, fine-tuned from pretrained weights, that infers UI element bounding boxes and a coarse 9-class taxonomy (`text`, `image`, `icon`, `button_labeled`, `button_icon_only`, `input_field`, `checkbox_toggle`, `tab_item`, `list_item`) directly from screenshot pixels. Training labels were auto-generated from XML bounds at label-generation time only – the trained model never sees XML at inference. Key hyperparameters: 60 epochs (early-stop patience 10), batch size 8, image size 960 px (to preserve small mobile UI elements), cosine learning-rate schedule, and vertical flip/rotation augmentation disabled, since mobile screenshots are never inverted or tilted. A `detections_to_components()` converter maps raw detections into `components.json`-shaped entries explicitly

TABLE IV: YOLO11s UI-element detector results.

Metric	MASC val (epoch 60)	
Precision	0.535	
Recall	0.447	
mAP50	0.434	
mAP50-95	0.322	
Rico holdout (unseen)	Zero-shot	Fine-tuned
mAP50	0.0258	0.2546
mAP50-95	0.0116	0.1732

tagged `inferred: true`, so downstream rule confidence can be adjusted and the origin of the data is never silently hidden.

Figures 4–8 present the full training and evaluation artifacts generated by the 60-epoch run. Figure 4 is Ultralytics’ auto-generated label-statistics summary of the training set itself – per-class instance counts and the spatial/scale distribution of ground-truth boxes – confirming the expected class imbalance (`text` and `icon` dominate) that motivated the per-rule handling elsewhere in the pipeline. Figure 5 is Ultralytics’ combined results plot: box/classification/DFL loss and precision/recall/mAP curves over all 60 epochs for both the train and validation splits, showing losses still trending down and mAP climbing smoothly with no divergence. Figure 6 is a compact, custom-rendered summary of the same 60-epoch run generated directly from `results.csv` for quick inspection outside the Ultralytics report. Figure 7 shows the four confidence-threshold sweep curves – Precision, Recall, F1, and Precision–Recall – broken down per class, useful for choosing an operating confidence threshold per UI-element type. Figure 8 shows the raw and row-normalized confusion matrices across all nine classes plus background, which surfaces the model’s dominant confusions (chiefly `text` vs. `button_labeled`, and small icons collapsing into background at low confidence).

Table IV summarizes the final 60-epoch results on the MASC validation split alongside the Rico holdout generalization check.

Fine-tuning yields roughly a 10× lift in mAP50 over the untrained base model on apps never seen during training, confirming that the training signal transfers beyond MASC, although absolute accuracy on unseen layouts (mAP50 ≈ 0.25) remains moderate. The detector is wired into `backend/routers/audit.py`: the `xml` field on `POST /api/v1/audit` is optional, and `_run_pipeline` falls back to `detect_ui()` on missing, malformed, or empty-hierarchy XML. Figures 9 and 10 show the Axion Upload screen states that surface this fallback path to the user.

2) *Multi-Label Crop-Violation Classifier*: Six of the thirty rules (R09 contrast, R04 touch-target size, R17 spacing, R10 text overflow, R28 font-scale overflow, R08 layout overlap) describe properties that only fully exist once a layout is *rendered* – bounds math from the XML alone can approximate but not confirm them. `src/crop_violation_classifier.py` implements a multi-label image classifier – since one crop

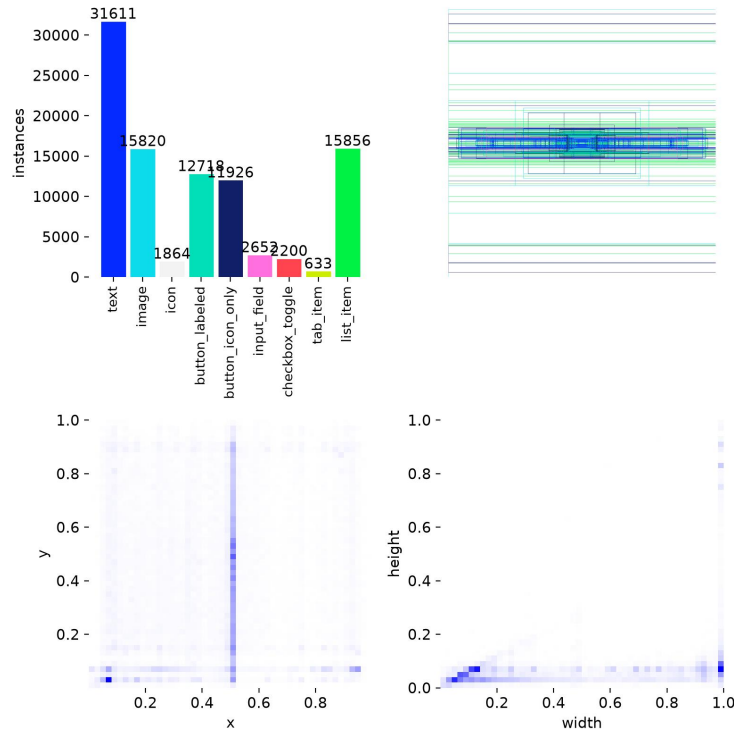


Fig. 4: YOLO11s training-set label statistics: per-class instance counts and the spatial (x,y) and scale (width, height) distribution of ground-truth boxes across the MASC train split, auto-generated by Ultralytics.

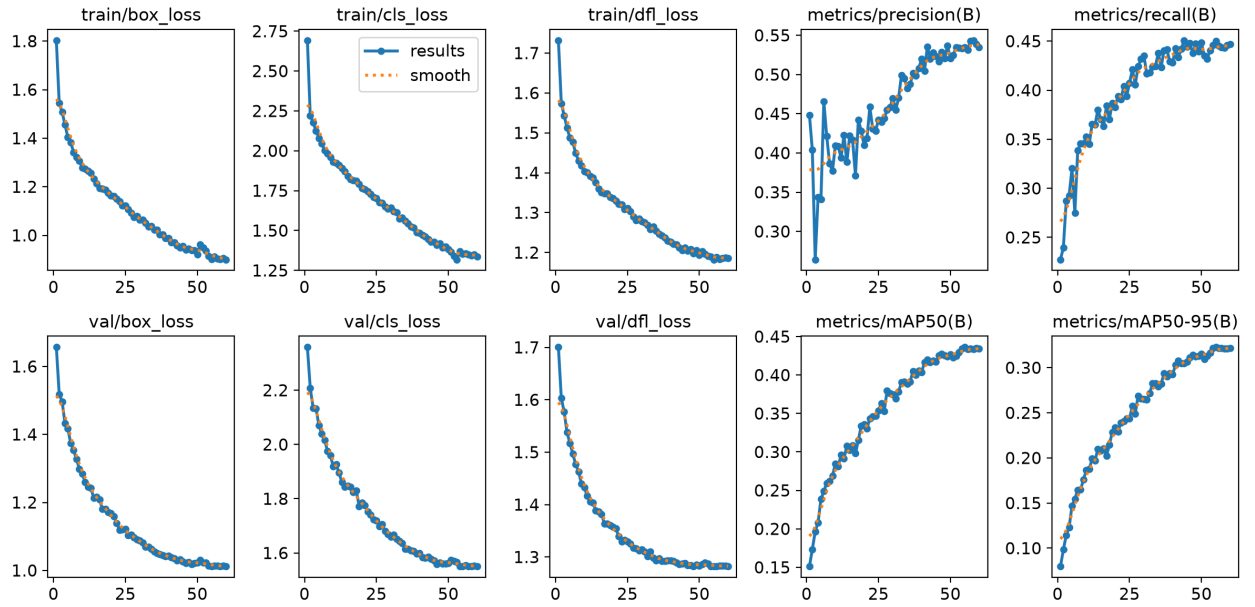


Fig. 5: YOLO11s training run: box/classification/DFL loss and precision/recall/mAP50/mAP50-95 over 60 epochs (train and validation splits), auto-generated by Ultralytics from `results.csv`.

can trigger more than one rule simultaneously, e.g., a control that is both too small and too close to its neighbor – that predicts a subset of $\{R09, R04, R17, R10, R28, R08\}$ for a 224×224 crop of a screenshot region. Positive/negative

crop labels are generated by re-running the already-tested rule checker itself on each MASC screen and keeping only the six target rules, avoiding a re-implementation of contrast/overlap/spacing math. Figure 11 shows representative

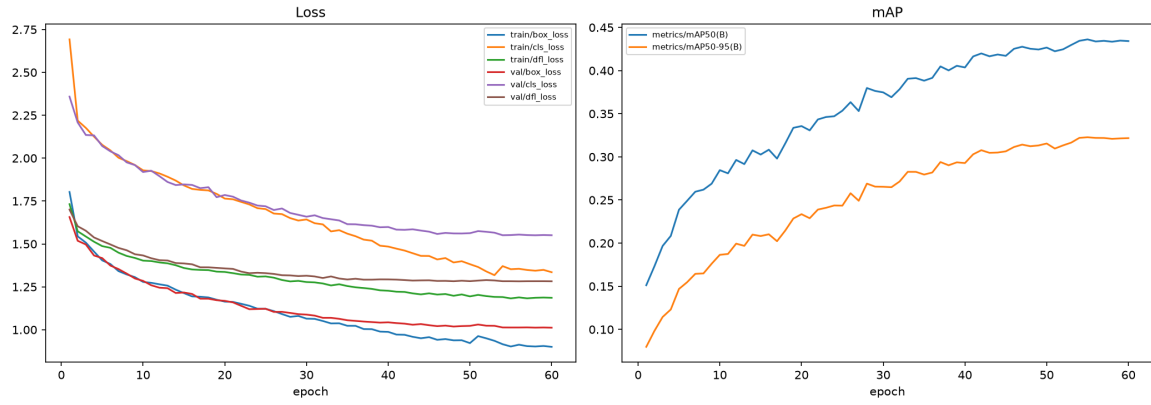


Fig. 6: YOLO11s detector: custom-rendered loss and mAP progression over the 60-epoch run, generated independently of the Ultralytics auto-report for quick visual sign-off.

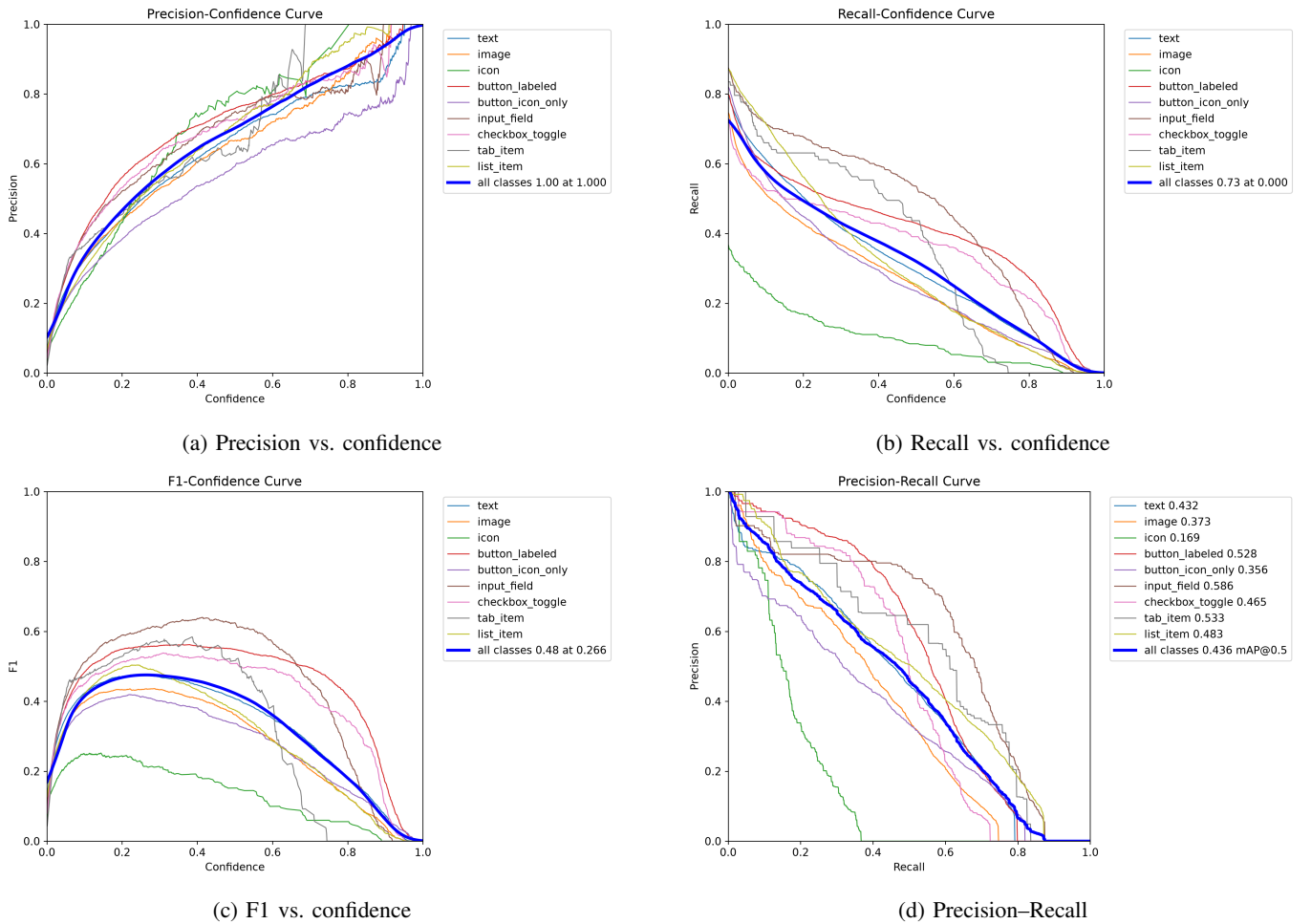
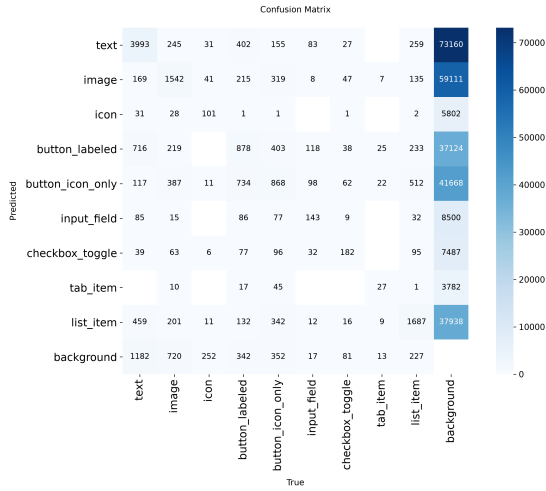


Fig. 7: YOLO11s per-class Precision, Recall, F1, and Precision-Recall curves on the MASC validation split.

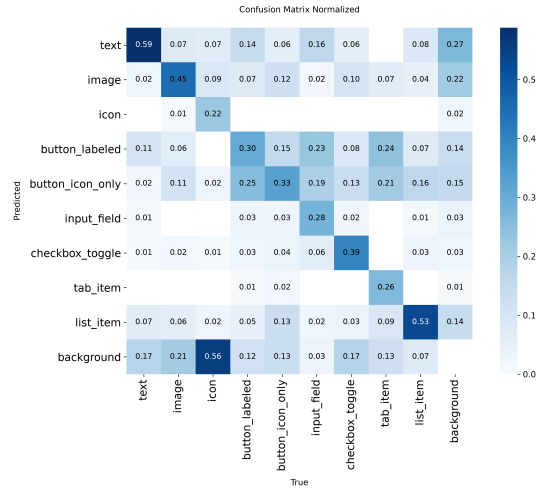
crops from the training set together with qualitative predictions on the MASC test split and the Rico holdout.

Training uses a two-phase fine-tuning schedule (frozen-backbone head-only for epochs 1–3, full unfreeze at a lower learning rate from epoch 4), a BCEWithLogitsLoss

with per-rule positive-class weighting capped at 20.0 to counter severe class imbalance, and early stopping on validation macro-F1 (patience 5). The original backbone choice, `mobilenet_v3_small`, was selected early for its low capacity relative to the available dataset size, but that



(a) Raw confusion matrix



(b) Row-normalized confusion matrix

Fig. 8: YOLO11s confusion matrices across the 9-class UI-element taxonomy plus background, MASC validation split.

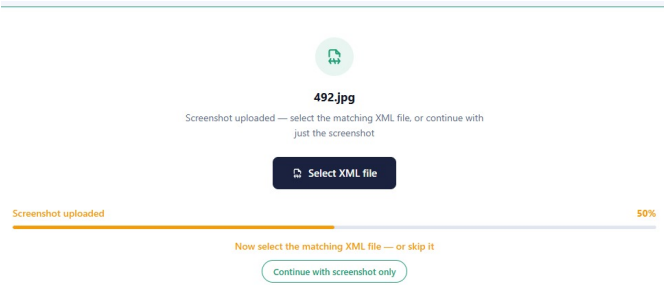


Fig. 9: Axion Upload: screenshot-only mode, awaiting the YOLO fallback detector while no XML is supplied.

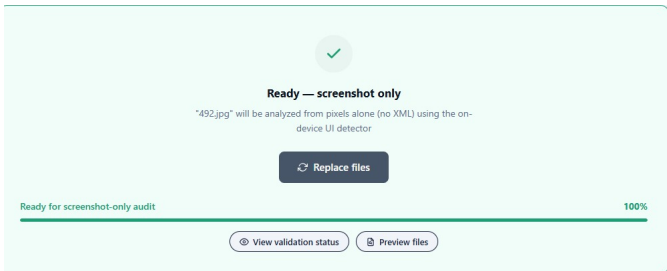


Fig. 10: Axion Upload: screenshot-only audit ready to start once pixel-based detection completes.

assumption was never empirically tested until a controlled 33-backbone comparison sweep was run against both the MASC test split and the full Rico holdout. The sweep found the original pick mid-pack (Rico macro-F1 0.18, beaten by ten or more candidates), and that attention-based architectures – not simply modern CNN design tricks – were driving the gap: a `convnext_tiny` control sharing the same design tricks (LayerNorm, patchify stem, inverted bottlenecks) but no attention

TABLE V: Crop-violation classifier (Swin-Tiny) results by rule.

Rule	MASC test F1	Support	Rico F1	Support
R04	0.31	45	0.40	145
R17	0.49	660	0.38	1,419
R08	0.72	1,805	0.51	3,418
R09/R10/R28	—	0	—	0
Macro avg.	0.25	2,510	0.22	4,982

landed back at pack level (0.18), while `vit_b_16` (global self-attention), `mobilevitv2_200` (hybrid CNN/self-attention), and `swin_tiny_patch4_window7_224` (windowed/hierarchical self-attention) all scored higher, with Swin-Tiny winning outright at roughly a third of ViT-B/16’s parameter count and a more evenly distributed per-rule profile. The backbone was consequently changed to `swin_tiny_patch4_window7_224`.

Figure 12 shows the resulting training curves; the frozen-backbone phase plateaus at val macro-F1 ≈ 0.20 before the backbone unfreeze produces a clear +37% relative jump to a best of 0.274 at epoch 8, after which the model oscillates and early stopping triggers at epoch 13 (only the best-val-F1 checkpoint is ever kept). Figure 13 and Figure 14 give the per-rule confusion matrices and threshold-sweep curves for R04/R17/R08, the three rules with real positive training signal. Table V reports the final classification results on both the MASC test split and the Rico holdout.

R08, the rule with by far the most positive training examples, is the strongest signal (F1 0.72 on MASC test, 0.51 on Rico); R17 is usable (0.49/0.38); R04 is data-starved but still generalizes reasonably (0.31/0.40); R09, R10, and R28 report zero support because neither MASC nor Rico ever declares the raw color or text-size attributes those rules depend on – a data-coverage gap in the source

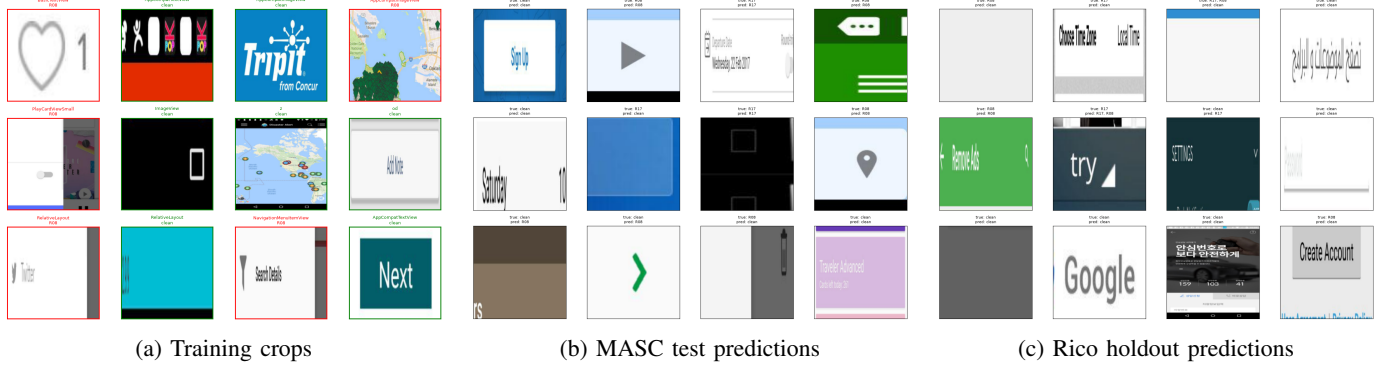


Fig. 11: Crop-violation classifier: sample positive/negative training crops, and qualitative multi-label predictions on unseen MASC test and Rico holdout screens.

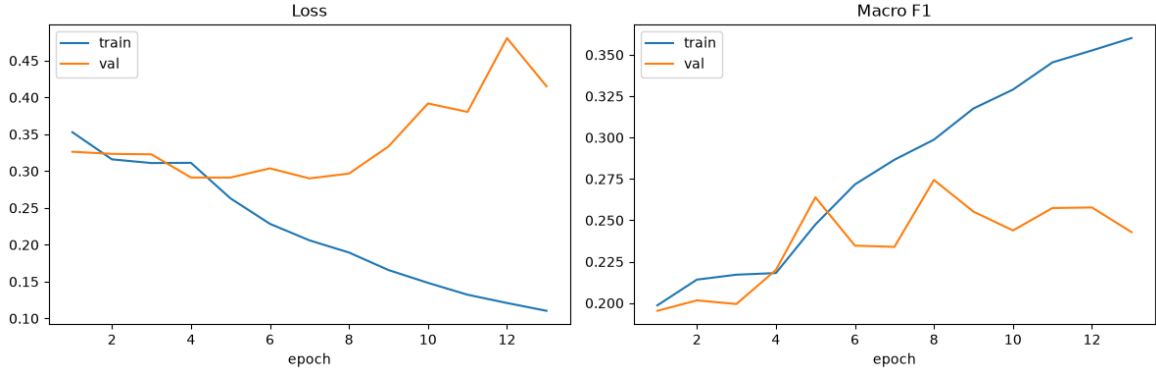


Fig. 12: Crop classifier (Swin-Tiny) training curves: two-phase fine-tuning schedule showing the frozen-backbone plateau (epochs 1–3, val macro-F1 ≈ 0.20) followed by a +37% relative jump after unfreezing (epoch 4 onward, peaking at 0.274 at epoch 8) before early stopping at epoch 13.

corpora, not a modeling failure. Rico macro-F1 (0.22) was the best result across the entire 33-backbone sweep and showed the tightest MASC-to-Rico generalization gap among top performers, consistent with the sweep’s broader finding that attention-based backbones generalize measurably better than pure-CNN candidates on unseen apps. The classifier is wired into backend/routers/audit.py: `confirm_violations()` runs immediately after rule checking and attaches an optional `cv_confidence` score (0–1) to eligible R08/R17/R04 violations without ever overriding or removing a rule-detected violation. Figure 15 shows how this confidence value surfaces in the Axion UI.

E. Agentic Explanation Layer

`src/agent.py` and `src/explainer.py` implement the LLM-driven enrichment stage. For every violation already present in `violations.json`, the agent generates three fields: a one-to-two sentence plain-English `agent_explanation`, an `agent_why_it_matters` statement describing impact on users with disabilities, and a concrete one-to-three line `agent_developer_fix` (e.g., an XML attribute change). Listing 2 shows the structure of the per-violation prompt template.

Listing 2: Per-violation agent prompt template.

```
System: Explain accessibility violations only.
       Never invent new violations.

User:
  rule_id: {rule_id}
  issue: {issue}
  component: {class, bounds, resource_id}
  guideline: {guideline}
  recommendation: {recommendation}

Return JSON: {agent_explanation,
              agent_why_it_matters,
              agent_developer_fix}
```

The system prompt explicitly instructs the model never to fabricate violations beyond those supplied, and the enrichment step is batched with an anti-hallucination match step that verifies returned explanations correspond to real input violation IDs. `src/llm_providers.py` abstracts over four providers (Anthropic, OpenAI, Gemini, Groq) behind a common interface selected via the `LLM_PROVIDER` environment variable; the deployed default is **Groq** (llama-3.3-70b-versatile), chosen after Anthropic and OpenAI were found to require paid billing and Gemini’s free tier hit a quota bug during development. API timeouts or errors fall back to template-based explana-

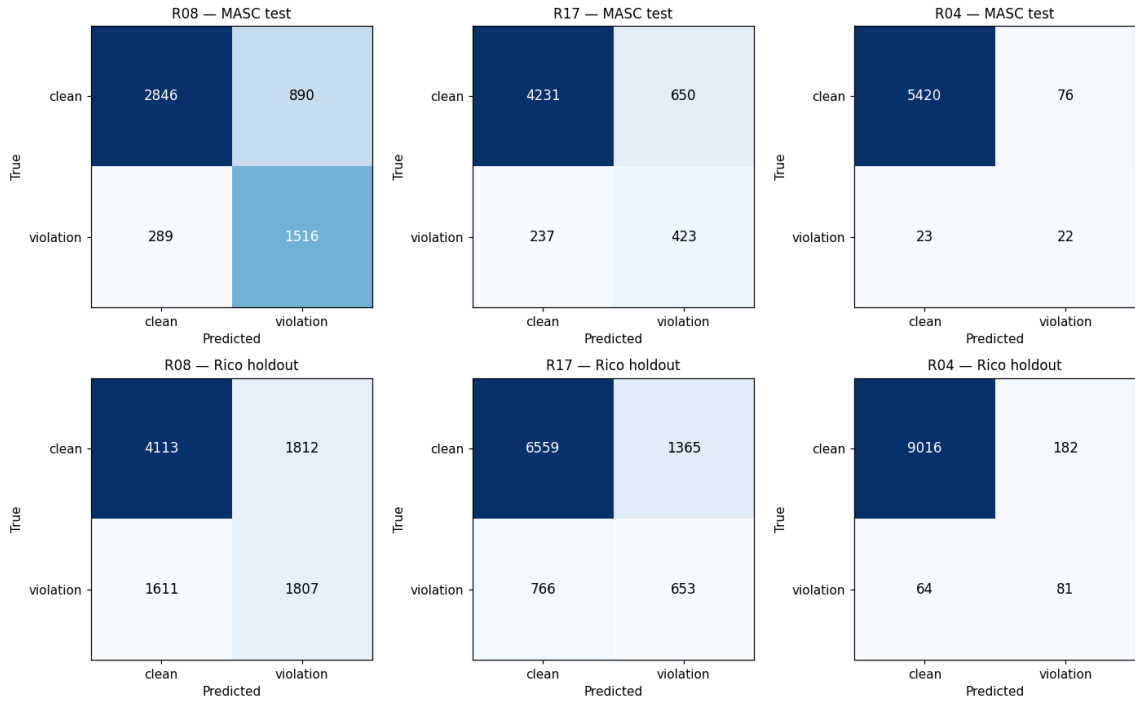


Fig. 13: Per-rule confusion matrices (R04, R17, R08) for the Swin-Tiny crop classifier on the MASC test split.

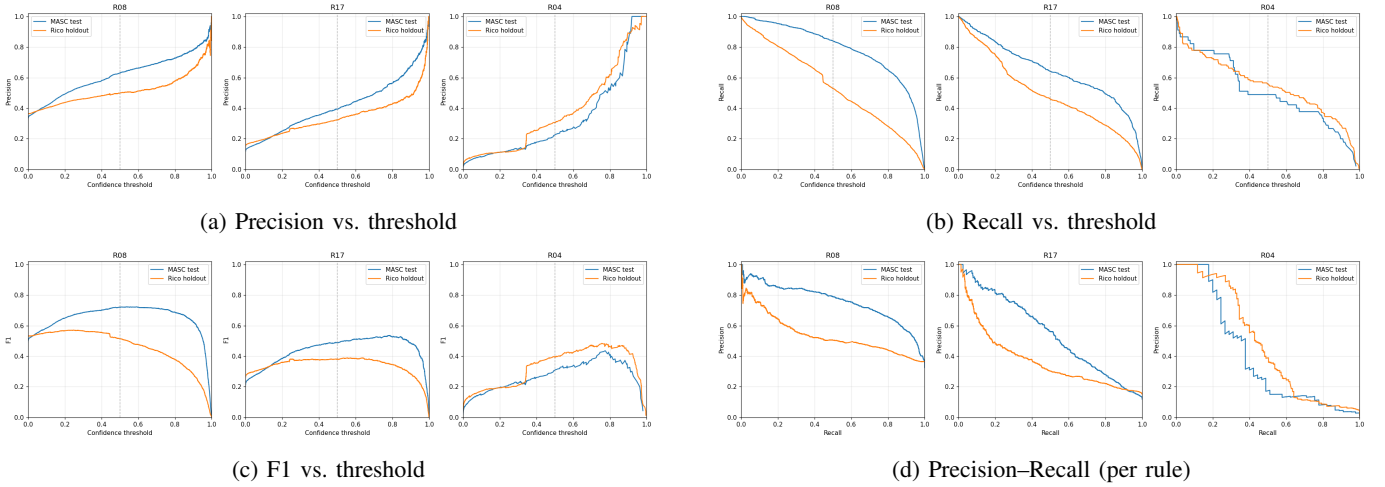


Fig. 14: Swin-Tiny crop classifier per-rule Precision, Recall, F1, and Precision-Recall curves, generated by `scripts/plot_crop_classifier_curves.py` in the style of Ultralytics’ auto-generated YOLO plots.

tions built directly from the rule engine’s own issue and recommendation fields, so the pipeline never blocks on a network failure, and the LLM API key is used server-side only and never exposed to the browser.

F. Report Generation

`src/report.py` renders the enriched `report.json` into both an HTML report (Jinja2 template, with PIL-drawn bounding-box annotations on the screenshot) and a PDF export (the same HTML rendered to PDF via headless Chromium through Playwright), exposed via `GET /api/`

`v1/audit/{id}/report/download?format=html|pdf`. Both formats include a cover section (screen ID, audit date, accessibility score, total violation count), a severity-ordered issue list (Critical/High first), and, per violation, the rule ID, WCAG reference, severity, affected component, agent explanation, impact statement, and developer fix. The 0–100 accessibility score is computed as

$$\text{score} = \text{clamp}(100 - (20 \cdot n_{\text{crit}} + 10 \cdot n_{\text{high}} + 5 \cdot n_{\text{med}} + 2 \cdot n_{\text{low}}), 0, 100) \quad (1)$$

and drives the Dashboard’s circular score gauge. Figures 16

R01	Missing accessible label android.widget.RelativeLayout · c_037	Critical	G01 — Missing accessible label	View
R08	Possible layout overlap android.support.v7.widget.x · c_013 CV-confirmed 95%	Serious	G08 — Possible layout overlap	View
R15	Bad focus order android.support.design.widget.TabLayout · c_015	Serious	G15 — Logical focus order	View

Fig. 15: Axion issue detail view showing a rule-detected violation annotated with a computer-vision confidence badge.

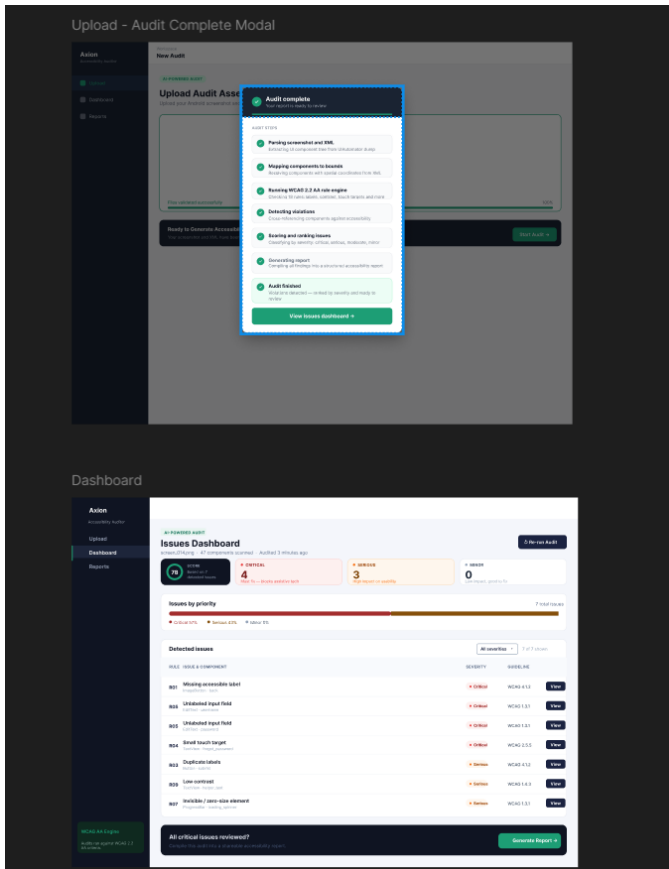


Fig. 16: Audit Complete modal (pipeline step checklist) and the Issues Dashboard: score gauge, severity summary cards, issues-by-priority bar, and violations table.

and 18 show the resulting Dashboard, Issue Detail drawer, Audit Report page, and the Generate Report modal with its PDF layout.

G. Frontend: Axion

The React 19 + Vite + Tailwind CSS frontend, designed first in Figma and implemented to match, provides Sign Up / Log In / Forgot Password flows (Figures 19 and 20), a sequential Upload flow, an Issues Dashboard with a slide-in Issue Detail drawer, an Audit Report screen with a Generate Report modal,

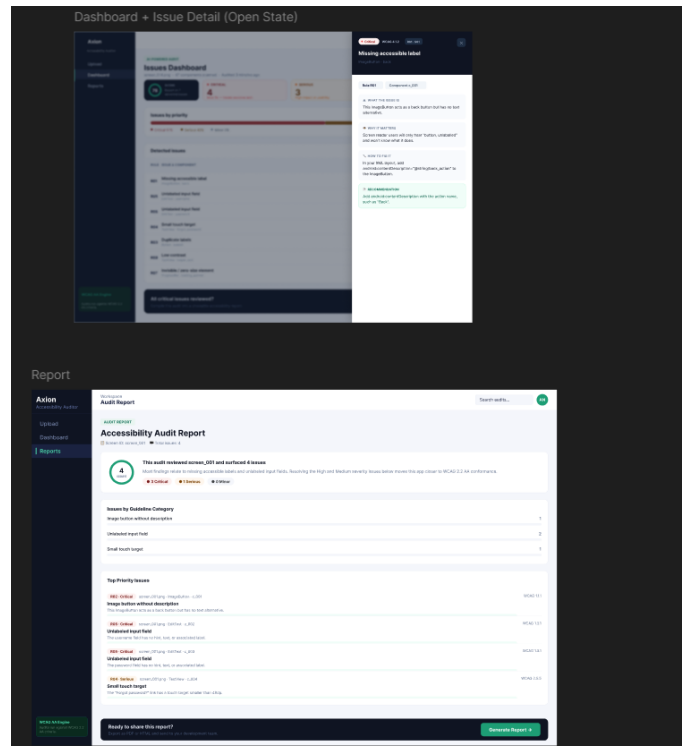


Fig. 17: Issue Detail drawer (what the issue is / why it matters / how to fix it) and the Audit Report page with guideline-category breakdown.

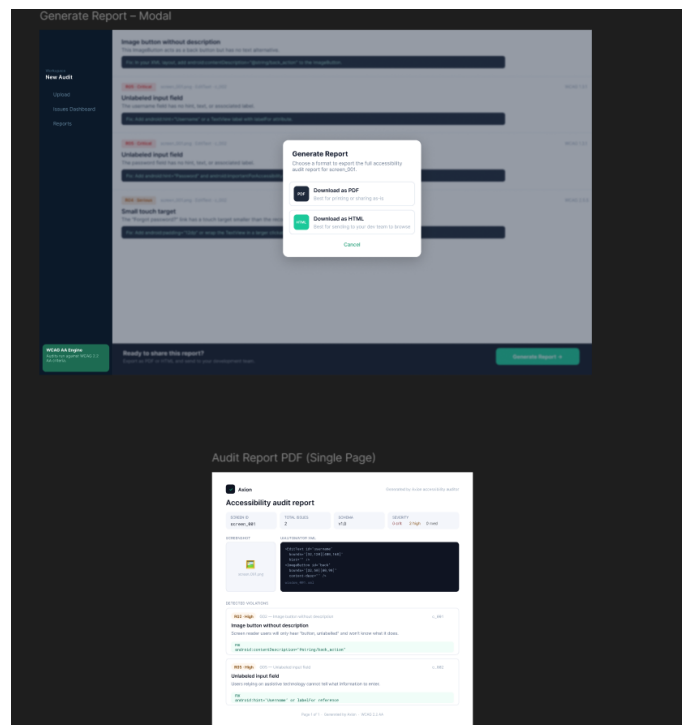


Fig. 18: Generate Report modal (PDF/HTML download options) and the resulting exported PDF layout.

TABLE VI: Security threats and mitigations.

Threat	Mitigation
API key exposure	LLM calls made server-side only
JWT theft	HTTPS in production; httpOnly cookie option (Stretch)
XXE injection	Safe XML parsing, no external entity resolution
Upload abuse	20 MB per-file limit, MIME validation
Path traversal	Sanitized paths in Records service
Cross-user access	JWT <code>user_id</code> scoping on every records query
OTP brute force	Rate limiting and expiry countdown

and a Records page listing every past audit for the logged-in user with score, date, and severity summary. All authenticated screens share a persistent sidebar (logo, navigation, WCAG AA Engine badge) and a consistent dark-navy/teal (#0b1929/#1bc99a) visual identity, with primary action buttons kept always visible rather than hover-dependent – a deliberate accessibility choice in the auditor’s own interface.

H. Authentication and Records

User accounts are backed by JWT sessions, with registration and login accepting any valid email domain, a forgot-password flow that emails a six-digit, time-limited OTP via SMTP with resend support, and Google Sign-In via server-side ID-token verification against a configured `GOOGLE_CLIENT_ID`. Every completed audit is persisted against the authenticated `user_id` in a flat JSON store (`backend/data/records.db.json`) recording screen ID, timestamp, violation counts by severity, accessibility score, original filenames, and artifact paths, exposed through user-scoped `GET/POST/DELETE /records` endpoints that never leak another user’s records. This behavior was not always correct in practice: the frontend call that persists a completed audit had been documented as implemented since the design specification’s first version but was never actually wired up, so audits completed and displayed correctly while silently never appearing in the user’s history – caught only by comparing the design documentation against the running frontend rather than by inspecting either in isolation, and fixed by adding the missing call with a duplicate-record guard. Table VI summarizes the security threats considered and their mitigations.

I. Deployment

The system ships as a three-service Docker Compose topology – `backend` (FastAPI + pipeline, port 8000 internally), `auditor` (a batch worker running the parser/rule pipeline over a mounted dataset), and `frontend` (Vite dev server, port 5173) – on an internal bridge network, started with a single documented command (`docker compose up --build`). Two real deployment bugs were found and fixed by actually building and running the containers rather than assuming the compose file was correct: the `backend`’s `requirements.txt` was missing the `torch/torchvision/timm/ultralytics` dependencies pulled in by the two CV modules, causing a startup crash, and after that fix the container still failed with `ImportError:`

`libxcb.so.1` because `ultralytics` depends on the full GUI-capable `opencv-python` rather than the headless variant, requiring several system libraries (`libgl1`, `libglib2.0-0`, `libsm6`, `libxext6`, `libxrender1`) not present in the base `python:3.11-slim` image.

A further, structurally similar bug surfaced only when deploying to a second, GPU-equipped machine: the `backend` image’s build context had been the `backend/` subdirectory, so it never actually packaged the repository-root `src/` and `models/` directories that the pipeline imports at runtime, working locally only because development compose bind-mounts the whole repository over the container at runtime. A standalone container built from that image and run without the bind mount – exactly the deployment scenario on the second machine – failed immediately with a missing-module error. The fix changes the build context to the repository root and copies `src/`, `models/`, and `backend/` explicitly, making the image genuinely self-contained. The `backend` service also gained an NVIDIA GPU reservation (Docker Compose’s `deploy.resources.reservations.devices`), and its externally published port was made configurable (8001 rather than 8000, with the container’s internal port unchanged) to accommodate the target lab machine’s port 8000 already being in use by unrelated services – both changes verified by a full audit, including the screenshot-only detection path, completing successfully end to end on that machine with GPU acceleration active.

J. Testing

The complete pytest suite comprises 155 tests spanning the parser (including MASC wrapper-nesting edge cases), the R01–R30 rule engine against 61 controlled pass/fail XML fixtures, the agent’s score formula and template fallback, the report generator’s HTML/PDF export, the FastAPI audit routes (including CV-wiring and YOLO-fallback behavior), and a dedicated 22-test authentication suite (JWT, OTP, Google OAuth, records isolation). Integration-level sign-off scripts additionally re-run the full pipeline against the entire MASC corpus and the Rico holdout after every significant rule or parser change, giving dataset-scale regression coverage beyond the unit fixtures.

VII. EVALUATION METHODOLOGY

Evaluation combined several complementary methods, aligned with the project’s QA test plan: (1) 10–15 controlled XML fixtures with known expected violations, used as unit-level regression tests for R01–R05 and beyond; (2) full-pipeline runs across 25–40 curated screens with manual labeling of each detected violation as Correct / False Positive / Missed (Week 6, 40 stratified-random screens – four per MASC category, seed 20260715 for reproducibility); (3) rule-wise and guideline-wise aggregate QA summaries across a larger MASC sample and the full Rico holdout (Week 7); and (4) a final batch evaluation of the entire 1,698-screen Rico holdout with zero pipeline failures, giving both a health

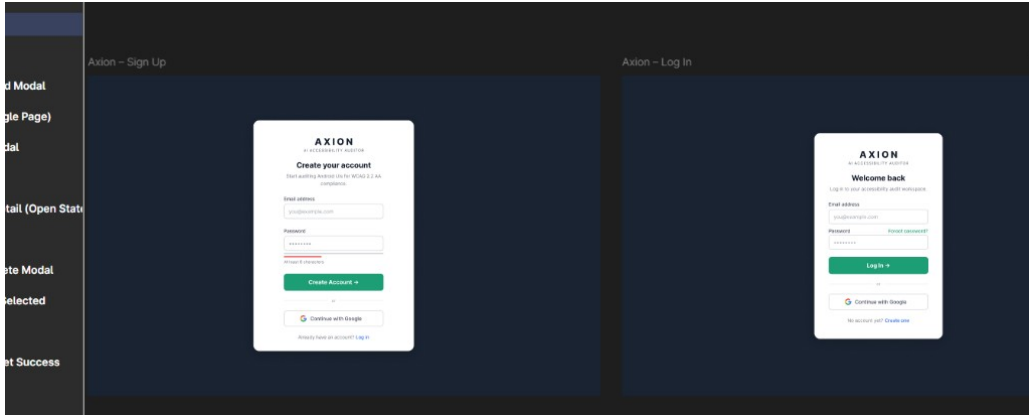


Fig. 19: Axion Sign Up and Log In screens, with password strength hints and Google Sign-In.

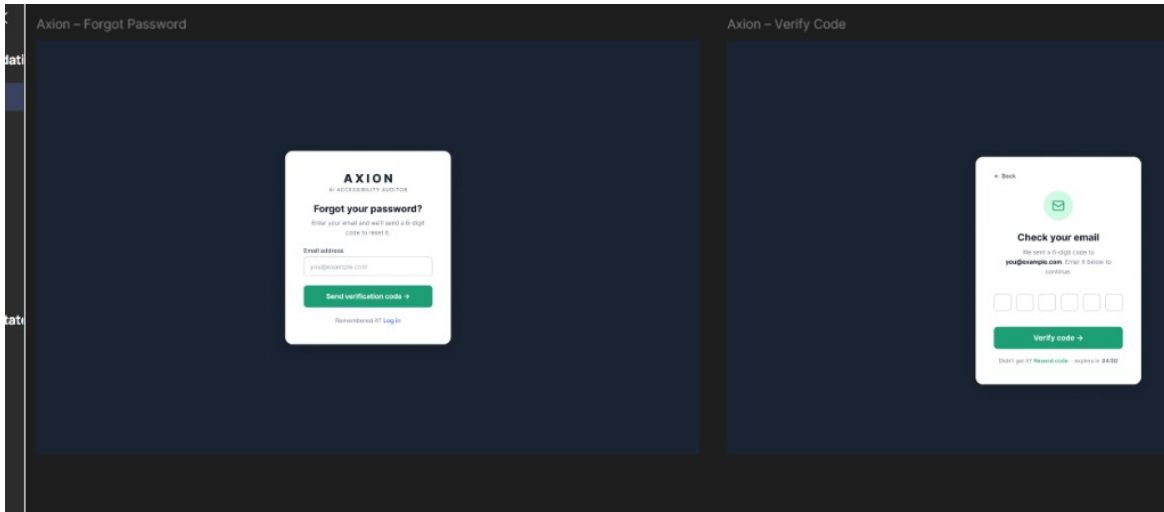


Fig. 20: Forgot Password flow: email entry and six-digit OTP verification with a resend countdown.

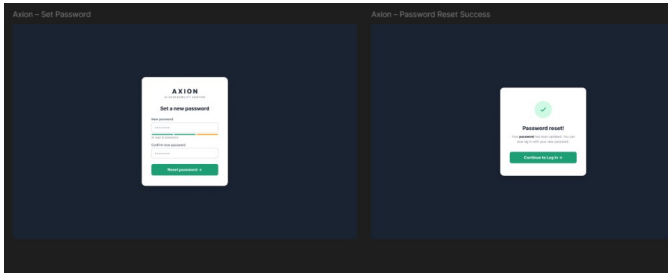


Fig. 21: Set Password screen and password-reset success confirmation.

snapshot of the rule engine at scale and a genuine unseen-data generalization check for the two CV models.

VIII. RESULTS

A. Manual Review (Week 6, 40 Screens)

Across the 40 stratified-random MASC screens manually reviewed, 35 of 40 were classified `mostly_agree` or `agree_clean` between the rule engine’s findings and human

judgment, with the remaining 5 flagged `over_flagging` or `mixed_r30_noise`. Three recurring false-positive themes were identified – R07/R08 nesting of a parent clickable container against children whose text already names the control, borderline contrast flags on secondary chrome elements by design, and R30 density from scroll-list icon repeats – all three of which motivated the Week 8 rule-accuracy fixes described above. Recurring miss themes (issues real but outside the static XML’s reach) included live/dynamic announcements such as chat unread badges, field-error association on auth forms, and selected-state on bottom navigation tabs when the XML lacks state attributes.

B. Rule Engine at Scale (Rico Holdout)

Table VII summarizes the aggregate health of the rule engine over the full 1,698-screen Rico holdout, run after the Week 8 accuracy fixes.

Table VIII lists the most frequently triggered guidelines across the holdout by screen coverage. Missing/absent accessible labels (G01/G02/G30) and zero-area or invalid-bounds elements (G07/G25, which share detection logic) dominate,

TABLE VII: Rico holdout aggregate rule-engine results (1,698/1,698 screens, 0 failures).

Metric	Value
Mean violations / screen	31.7
Median violations / screen	14
Mean accessibility score (0–100)	32.0
Screens with zero violations	87

TABLE VIII: Top guidelines by screen coverage, Rico holdout (1,698 screens).

ID	Guideline (short)	% screens
G01/G02/G30	Missing / absent accessible labels	85.3%
G07/G25	Zero-area or unpausable/invalid elements	63.5%
G18	Multi-finger-only interaction	47.0%
G16	Decorative element left in focus tree	49.5%
G15	Focus order not top-to-bottom	46.4%
G08	Overlapping content/focus order	27.5%
G17	Insufficient touch-target spacing	28.4%

appearing on 63–85% of screens; several guidelines that depend on declared color or text-size attributes (G09, G10, G22, G28, G29) never fire on either corpus, a data-coverage limitation of the source XML rather than a rule defect.

The most frequently triggered individual rules were R07 (zero/invalid-bounds elements, 63.5% of screens), R01 (missing accessible label, 85.3% of screens), and R08 (layout overlap, 27.5% of screens); R09, R10, R20, R22, R25, R28, and R29 never fired on the holdout, consistent with the underlying data-coverage gap rather than a rule-logic failure. Manual QA classified the dominant residual noise pattern as clickable-container-vs-child overlap and repeated icon instances – exactly the two patterns the Week 8 R08/R30 fixes targeted, confirmed by the -43.2% R08 reduction reported in Section V-C.

C. Computer-Vision Modules

Both stretch computer-vision modules were trained to completion, evaluated on the unseen Rico holdout, and wired into the production backend pipeline rather than left as standalone notebooks. The YOLO detector (Table IV, Figures 5–8) demonstrated a genuine, if moderate, generalization signal – roughly a 10× mAP50 lift over its untrained base on unseen apps. The Swin-Tiny crop classifier (Table V, Figures 12–14) achieved its best Rico holdout macro-F1 (0.22) of any of the 33 backbones evaluated in the comparison sweep, with the tightest MASC-to-Rico generalization gap among the top performers.

D. System-Level Validation

The complete pytest suite (155 tests) passed in full. All 99 functional and non-functional requirements tracked in the project’s requirements traceability matrix were marked Done by project close, including both Stretch computer-vision integrations, which were initially deferred and completed only in the final week of the internship.

IX. DISCUSSION AND LIMITATIONS

Several limitations are worth stating explicitly rather than glossing over. First, the primary rule-detection accuracy is bounded by what the source datasets actually declare: MASC and Rico XML dumps never populate the raw color or text-size attributes that R09 (contrast), R28 (font-scale overflow), and R29 (all-caps body text) depend on, so those rules are correctly implemented but structurally inactive on the available corpora – a data-coverage gap rather than a code defect. Second, both computer-vision modules, while genuinely wired into the pipeline and showing real (not memorized) generalization to the unseen Rico holdout, report modest absolute accuracy (macro-F1 0.22, mAP50 0.25); they are positioned as supplementary confidence signals that never override the deterministic rule engine, not as a replacement for it. Third, the crop classifier could only be trained with real positive examples for R08, R17, and R04 – R09/R10/R28 have zero positive training crops for the same data-coverage reason described above, so `confirm_violations()` deliberately restricts itself to the three rules with genuine signal. Finally, the prototype’s persistence layer is a flat JSON store rather than a production database, and its authentication/authorization is JWT-based rather than enterprise SSO – both explicit, documented scope decisions appropriate for an eight-week research prototype rather than a production system.

X. FUTURE WORK

Directions identified but not pursued within the internship’s scope include: a dedicated pixel-sampling contrast module that measures rendered foreground/background colors directly from the screenshot rather than relying on declared XML attributes, closing the R09 data-coverage gap independently of dataset limitations; specialized detectors for visual text clipping to give R10/R28 a real signal; interactive click-to-highlight linking between the HTML report and the screenshot; a curated, manually validated benchmark dataset released as a project deliverable; and batch/CLI evaluation tooling suitable for continuous-integration regression testing of the rule engine against the full MASC and Rico corpora on every code change.

XI. CONCLUSION

This internship delivered a complete, end-to-end Agentic Accessibility Auditor for Android UIs: a deterministic 30-rule engine mapped to 30 WCAG-aligned guidelines, an LLM-based explanation layer constrained to never fabricate findings, an HTML/PDF report generator, a fully authenticated React dashboard with per-user audit history, and – beyond the original Must-Have scope – two trained and backend-integrated computer-vision models providing a screenshot-only fallback detector and pixel-level rule confirmation. The system was developed against a 7,068-screen dataset and validated, without tuning, on a disjoint 1,698-screen holdout, with all 99 tracked requirements and 155 automated tests passing at project close. The result is a practical demonstration that combining a deterministic, auditable rule engine with a

tightly scoped LLM explanation layer can produce accessibility tooling that is both more thorough than screenshot-only scanners and more explainable than a black-box classifier.

ACKNOWLEDGMENT

The authors thank their team lead and the FAST-NUCES internship supervisors for guidance throughout the eight-week program, and acknowledge the creators of the MASC and Rico datasets, without which the parser, rule engine, and computer-vision modules described here could not have been developed or evaluated.

REFERENCES

- [1] World Wide Web Consortium (W3C), “Web Content Accessibility Guidelines (WCAG) 2.2,” W3C Recommendation, 2023.
- [2] Android Developers, “Write automated tests with UI Automator,” Google, 2026. [Online]. Available: <https://developer.android.com/training/testing/other-components/ui-automator>
- [3] IEEE, “IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications,” Institute of Electrical and Electronics Engineers, 1998.
- [4] G. Jocher et al., “Ultralytics YOLO11,” Ultralytics, 2024.
- [5] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, “Swin Transformer: Hierarchical Vision Transformer using Shifted Windows,” in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2021, pp. 10012–10022.
- [6] B. Deka et al., “Rico: A Mobile App Dataset for Building Data-Driven Design Applications,” in *Proc. 30th Annual ACM Symp. User Interface Software and Technology (UIST)*, 2017.
- [7] A. A. Ahmed, “MASC Dataset,” Kaggle, 2023. [Online]. Available: <https://doi.org/10.34740/kaggle/ds/3962521>
- [8] Kaggle user onurgunes1993, “RICO Dataset,” Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/onurgunes1993/rico-dataset>
- [9] M. Zhong, R. Chen, X. Chen, J. Fogarty, and J. O. Wobbrock, “ScreenAudit: Detecting Screen Reader Accessibility Errors in Mobile Apps Using Large Language Models,” in *Proc. ACM CHI Conf. Human Factors in Computing Systems (CHI)*, 2025.
- [10] J. O. Ansah, A. Kapoor, A. Khanna, M. Vinod, P. Njeck, and S. Gao, “Insight: Enhancing Mobile Accessibility for Blind and Visually Impaired Users with LLMs,” arXiv:2605.09803, 2026.
- [11] S. Ramirez, “FastAPI Documentation,” 2024. [Online]. Available: <https://fastapi.tiangolo.com>
- [12] Meta Platforms, “React 19 Documentation,” 2024.